

Design and Implementation of a Web-Based, End-to-End Encrypted Secure Storage Service

Università di Bologna – Cybersecurity

Amadeus Beckmann (0001185880)

9. February 2025

Abstract

Increasing reliance on digital technologies has amplified concerns about data security and privacy, particularly as cloud services expose sensitive information to risks such as unauthorized access and potential misuse in AI training. This report presents the design and implementation of a web-based, end-to-end encrypted secure storage service that addresses these challenges by ensuring data confidentiality and user privacy. Using state-of-the-art cryptographic techniques, including XChaCha20-Poly1305 and Curve25519, the system provides secure file storage and retrieval. Key features of the application are client-side encryption, content-defined chunking for efficient data transmission, and *public vaults* for collaboration. While the implementation based on SvelteKit, gRPC, and SQLite is rather simple, it already shows the potential and ease of client-side encryption. In addition to presenting the cryptographic design and implementation details, this thesis evaluates the risks posed by the system to identify potential vulnerabilities by applying a cybersecurity risk analysis. This proof of concept demonstrates how cryptographic best practices can be applied to build secure storage solutions, contributing to the broader discourse on cybersecurity and privacy in the digital age.

1 Introduction

The rapid advancement of digital technologies has brought unprecedented opportunities for data sharing and collaboration but also introduced significant risks to data security and privacy. In particular, the growing use of artificial intelligence (AI) poses a new dimension of risk to personal data, as information stored in cloud services can potentially be used to train AI models, without the explicit consent of users [1]. But it is not only data misuse by visible actors or server operators that jeopardizes privacy, but also the growing danger of becoming a victim of cybercriminals [2].

Therefore, cybersecurity has become a critical concern for individuals, companies, and governments, and is driven by the increasing frequency of data breaches, unauthorized access, and exploitation of sensitive information. Protecting data before uploading it to possibly insecure servers, during transit, and at rest requires robust cryptographic methods to ensure confidentiality, integrity, and authenticity.

This report contributes to the field of cybersecurity by investigating the design and implementation of a web-based, secure storage service that encrypts data, before any server even sees it. This ensures that only authorized users can access data, even if the storage server is compromised. This approach

addresses core cybersecurity principles, such as minimizing attack surfaces and safeguarding user privacy [3], particularly in the context of modern threats like data misuse in AI training and cloud vulnerabilities. In the opaque world of cloud providers, there is no security for private data and without client-side encryption, users are left to trust that providers will respect their customers' privacy. Only if data is securely encrypted before it is stored remotely and potentially leaked from a server, privacy can be fully protected.

Through this work, we demonstrate how cybersecurity principles can be applied to build a secure, user-centric storage solution that aligns with contemporary cybersecurity challenges [3].

1.1 Motivation

The primary motivation for this work is to deepen my understanding of cybersecurity principles and practices, including their real-world application. Additionally, with this project I aim enhance my knowledge of cryptography, which is also a fundamental aspect of cybersecurity, by implementing and analyzing state-of-the-art cryptographic techniques.

1.2 Report Structure

The report is structured as follows. In Section 2, we briefly cover the most important concepts, which will certainly already be familiar to the experienced reader. While Section 3 outlines the scope of the developed application, Section 4 and Section 5 describe the cryptographic mechanisms behind the storage service and how the application is implemented respectively. Section 6 aims to identify potential risks and vulnerabilities. Finally, Section 7 concludes the report, including the discussion of limitations and potential future work in Section 7.1 and Section 7.2.

2 Background

In this section we will briefly cover the most important terms and concepts related to this work.

Symmetric cryptography. Symmetric cryptography uses the same secret key for both encryption and decryption [4]. This approach is generally faster than asymmetric methods and is commonly used to protect large volumes of data. A key challenge, however, is securely sharing the secret key among authorized parties. Common use of symmetric cryptography are secure data transfer (e.g. in TLS) and disk encryption.

Asymmetric cryptography. Asymmetric cryptography, also known as public key cryptography, uses two mathematically linked keys: one public key for encryption and one private key for decryption [4]. Anyone can have access to the public key, while the private key must remain confidential. This design simplifies key distribution but is typically slower than symmetric cryptography. Asymmetric cryptography is heavily used for secure key exchange, digital signatures, and encrypting emails.

Elliptic-curve cryptography. Elliptic-curve cryptography (ECC) is a specialized form of asymmetric cryptography. It relies on the mathematical properties of elliptic curves to offer strong security with smaller key sizes compared to older methods (like RSA) [4]. This also results in faster computations and reduced storage requirements.

Hashing algorithms. A hashing algorithm takes an input (such as a file or message) and outputs a fixed-size string of characters called a hash [4]. Good hashing algorithms are one-way (making it impractical to retrieve the original data from the hash) and collision-resistant (making it highly unlikely for two different inputs to produce the same output) [4]. Hashing is an important building block for verifying data integrity (message authentication), secure passwords storage, and digital signatures.

XChaCha20-Poly1305. XChaCha20-Poly1305 is an authenticated encryption scheme that extends the ChaCha20-Poly1305 scheme to use a larger (24-byte) nonce [5]. This larger nonce space helps prevent nonce reuse, which is critical for security. Like ChaCha20-Poly1305, it provides both confidentiality and

integrity (via the Poly1305 MAC). XChaCha20-Poly1305 is often used in modern cryptographic libraries (e.g., libsodium) for secure data encryption in applications such as messaging, file encryption, and network protocols. Although most modern CPUs don't have hardware support for XChaCha20-Poly1305, it has a fast software performance, and without hardware acceleration, is usually faster than AES-GCM [6].

Ed25519. Ed25519 is a high-performance digital signature algorithm using the Twisted Edwards curve Curve25519. Since it is designed to offer strong security and efficient verification, Ed25519 is widely used in secure messaging protocols, SSH keys, and other authentication mechanisms.

X25519. X25519 is an elliptic-curve Diffie-Hellman key exchange protocol based on Curve25519. It enables two parties to securely establish a shared secret over an insecure channel. X25519 is designed for both high performance and robust security. This makes it a popular choice in modern protocols such as TLS 1.3 and SSH.

End-to-end encryption. End-to-end encryption (E2EE) ensures that data is encrypted on the sender's device and can only be decrypted by the intended recipient. In E2EE applications, the encrypted data is safe from unauthorized access both during transit and at rest.

Key derivation function. A key derivation function (KDF) is used to produce new cryptographic keys from a shared secret or master key. KDFs typically combine hashing algorithms, salts (random data), and sometimes repeated iterations to generate stronger, unique keys for different encryption or authentication purposes. A popular key derivation function is PBKDF2, which applies a pseudo-random (such as HMAC) to a user-defined password along with a salt repeatedly to produce a *derived secret* which can then be used for subsequent cryptographic operations.

Content-defined chunking. Content-defined chunking (CDC) splits data into segments based on its actual content, rather than using fixed-size blocks. By analyzing and detecting boundaries within the data itself, this approach often improves storage efficiency, especially for data sets that experience frequent, incremental changes.

3 Application Scope

This section outlines the scope of the web-based secure storage service. While the planned functionality is relatively limited, it is already enough to provide a clear demonstration of how an end-to-end encrypted storage service can securely store and retrieve files and protect user privacy.

User registration. The first feature of our storage service is user registration. To keep things simple, registration requires only a username and a password. Usernames must be unique and passwords must have at least 8 character. Although technically possible, usernames and passwords cannot be changed after registration.

User login. Registered users can log in by entering their username and password. If the username does not exist or the password is incorrect, an error message indicates that the login attempt failed. Upon successful login, where both the username exists and the password is correct, the user should receive a signed session cookie. This cookie is necessary to authenticate future requests. The session cookie design also allows for the potential addition of session management features, giving users more control over active sessions and logged-in devices.

File upload. One of the core features of the web-based secure storage service is of course the file upload. Users can select a file from their local device, which is then encrypted locally before being uploaded to the server. To improve transmission resilience, files are split into smaller chunks. Each chunk is encrypted individually before it is sent to the server. Each client is granted a storage quota of 1 GB. Whenever an upload request would exceed this quota, an error is returned instead.

File listing. Users can view a list of all their stored files. The list of files allows them to perform further actions such as viewing a file, downloading it, or deleting it from their personal storage.

File download. The other core feature is file retrieval and decryption. The web application downloads all file chunks, decrypts them, and reassembles them in the correct order. The file can then be viewed in the browser or downloaded locally.

File deletion. The web-based secure storage service also includes a file deletion feature that allows authenticated users to delete unwanted files and free up storage space. As the file has previously been split into chunks, the client application has to identify the chunks belonging to the file to delete and then request file and chunk deletion for these assets.

Public vaults. The final feature of the storage service is “*public vaults*”. Authenticated users can create public vaults to generate a special URL they can send to third parties who can then upload files directly into the owner’s cloud storage. Only the owner of the public vault can read the contents of uploaded files. They may define slots within the vault, each with a custom name and a maximum upload size limit. When a third party uploads a file to a slot, the server enforces the defined size limit. Files can only be uploaded into empty slots, however, the vault owner can clear a slot at any time to make it available again. Public vaults can not be edited after creation, however, they can be deleted. Deleting a vault also deletes all associated files.

4 Cryptographic Design

The overall approach of this work is strongly inspired by MEGA, a web service for secure cloud storage, encrypted chats and meetings, and other security based tools. To enable users to understand and evaluate MEGA’s crypto design, the developers of MEGA have described their approach in a security white paper, i.e., how registration and login work, which data is encrypted and how, and which algorithms and parameters are being used [7].

Although MEGA’s cryptographic design has some clever mechanisms to ensure the integrity and security of user data, certain parameters of the underlying cryptographic functions are no longer up to date. For example, MEGA only uses 128 bits as the key length for the master key, the key which gives access to all other user data. While a shorter key length allows devices with less computing power to encrypt and decrypt files efficiently, this decision also jeopardizes data security for all users from the outset. Furthermore, to derive the master key from the user password, the PBKDF2-HMAC-SHA-512 algorithm is used with only 100,000 iterations, while OWASP’s recommendation is to use at least 210,000 iterations [8]. Finally, MEGA makes extensive use of AES in electronic codebook mode (AES-ECB), which has the disadvantage of lacking diffusion: it cannot hide data patterns when encrypting identical plaintext blocks into identical ciphertext blocks. Although MEGA uses AES-ECB only for very specific operations, the use of AES-ECB in cryptographic protocols is generally not recommended [9], [10], [11].

The approach chosen in this work attempts to replace these bad practices with best practices. For these means, it uses XChaCha20-Poly1305 instead of AES and X25519 instead of RSA.

Note: For better legibility the encryption function $XChaCha20-Poly1305(\dots)$ is abbreviated as $Enc(\dots)$.

Note: In the following subsections, the cryptographic design is explained in decreasing detail, as most of the ideas become clear through the first three subsections and in order not to exceed the maximum allowed length of 12 pages.

4.1 Registration Process

The registration process is closely based on the process described by MEGA [7], which itself is based on *Method to Protect Passwords in Databases for Web Applications* by S. Contini [12].

The registration process is as follows:

1. The user enters their **Username** and **Password**.
2. The client generates a random 32-bytes **Master Key**.
3. The client generates a 32-bytes **Client Random Value**.
4. The client computes a **Salt** using

$$\text{Salt} = \text{SHA-512}(\text{"tulip-"} \parallel \text{Padding} \parallel \text{Client Random Value}) ,$$

where **Padding** is the character “#” repeated, until the combined length of “tulip-” and **Padding** is 224 characters. Since the **Client Random Value** is exactly 32 bytes, the final input for SHA-512 has a length of 256 bytes.

5. Then, the **Password** and the **Salt** are passed into PBKDF2 to generate a derived key

$$\text{Derived Key} = \text{PBKDF2-HMAC-SHA-512}(\text{Password}, \text{Salt}, \text{Iterations}, \text{Length})$$

with **Iterations** set to 210,000 and **Length** set to 64 bytes.

6. The first half of the **Derived Key** (32 bytes) is called the **Derived Encryption Key** and is used to encrypt the **Master Key** using

$$\text{Encrypted Master Key} = \text{Enc}(\text{Derived Encryption Key}, \text{Master Key}, \text{Nonce})$$

with **Nonce** being a random 24 byte nonce. The nonce is then prepended to the encrypted data.

7. The second half of the **Derived Key** (32 bytes) is called the **Derived Authentication Key** and will be used to authenticate the user’s login before sensitive data (such as encrypted keys) are sent. The **Derived Authentication Key** functions as a password in traditional user authentication flows.
8. The client computes a **Hashed Authentication Key** by SHA-512 hashing the **Derived Authentication Key**:

$$\text{Hashed Authentication Key} = \text{SHA-512}(\text{Derived Authentication Key}) .$$

The **Hashed Authentication Key** is then shortened to the first 32 bytes.

9. Then, the client generates a **X25519 Private Key** and the according **X25519 Public Key** which are used for public vaults, and encrypts the **X25519 Private Key** using

$$\text{Encrypted X25519 Private Key} = \text{Enc}(\text{Derived Encryption Key}, \text{X25519 Private Key}, \text{Nonce})$$

Again, the **Nonce** is a random 24 byte nonce which is different from the nonce being used to encrypt the **Master Key**. Also, the nonce is prepended to the encrypted data.

10. The client then sends a register request to the server, containing the following data:

Username
Client Random Value
Encrypted Master Key
Hashed Authentication Key
Encrypted X25519 Private Key
X25519 Public Key

4.2 Login Process

In order for a user to log into their account and gain access to uploaded files, the following steps are performed:

1. The user enters their **Username** and **Password**.
2. The **Username** is sent to the backend.

- If the **Username** exists in the database, the server computes the **Salt** as it is done during registration:

Salt = SHA-512("tulip-" || **Padding** || **Client Random Value**) ,

where the **Client Random Value** belongs to the user and **Padding** is the character "#" repeated until the combined length of "tulip-" and **Padding** is 224 characters.

- If the **Username** is not found in the database, the server instead computes a *fake* **Salt** using

Salt = SHA-512(**Username** || "tulip-" || **Padding** || **Server Random Value**) .

For the padding, the character "#" is repeated until the combined length of **Username**, "tulip-", and **Padding** is 224 bytes, such that the combined length of the overall input is again 256 bytes.

Because length of the input for the SHA-512 function is 256 bytes for both valid and invalid usernames, the computation takes the same amount of time, which reduces the risk of timing attacks.

The server also waits for a random time between 0ms and 100ms before sending the **Salt** to the client. This helps to hide whether a username was valid or not from an attacker by measuring the delay in response from the database.

3. The client can then compute the **Derived Key**

Derived Key = PBKDF2-HMAC-SHA-512(**Password**, **Salt**, **Iterations**, **Length**)

using the same parameters as used in the registration process.

4. Again, the first half is used as the **Derived Encryption Key**.
5. The second half is used as the **Derived Authentication Key**.
6. The client then sends a login request to the backend with

Username
Authentication Key .

7. The backend then computes

Hashed Authentication Key = SHA-512(**Authentication Key**)

and searches for a user with the given **Username** and **Hashed Authentication Key**. If no matching user record was found, the server responds with an error that the user was not found. This response does not reveal whether the user simply does not exist or whether the authentication key was incorrect.

If the authentication was successful, the server responds to the client with the following data

Encrypted Master Key
Encrypted X25519 Private Key
X25519 Public Key .

In addition, the client receives a signed session cookie that contains a random 64-byte session identifier. This session identifier will be used for all further requests that require authentication. The session is then valid for 30 days or until the client logs out.

4.3 File Encryption

Each file is encrypted using a unique encryption key (and nonce). This is done in case the user wants to rotate the master key. In this case, only the keys of each stored file need to be re-encrypted instead of the entire file.

The detailed process is as follows:

1. The user selects a **File** to upload.
2. The client generates a random 32-byte **File Key**.

3. The client generates a random 24-byte **Base Nonce**.
4. The client initializes an empty array **Chunk IDs** which will contain the ids of the encrypted chunks stored on the server.
5. The client initializes a 16 byte long **Condensed MAC** with zeros. This MAC will later be used to obfuscated the **File Key**.
6. The **File** is then split into chunks.

1. Each chunk is encrypted using the **File Key** and a random, unique **Nonce**:

Encrypted Chunk = $Enc(\text{File Key}, \text{Chunk}, \text{Nonce})$.

Again, the nonce used for encryption is prepended to the encrypted data, yielding the **Encrypted Chunk**.

2. Since XChaCha20-Poly1305 is an authenticated encryption algorithm, the last 16 bytes of the **Encrypted Chunk** are the Poly1305 **Chunk Authentication Tag**.
3. In addition to encrypting the chunk, the **Condensed MAC** is updated by XORing the current **Condensed MAC** with the **Chunk Authentication Tag**. This intermediate result is then hashed using SHA-256 of which only the first 16 bytes are kept:

Condensed MAC = $SHA-256(\text{Condensed MAC} \oplus \text{Chunk Authentication Tag})[0..15]$.

4. The **Encrypted Chunk** is then sent to the backend which stores it and creates a database record, so that the chunk can be associated with the currently logged in user. The backend then sends the **Chunk ID** back to the client.
5. The client then appends this **Chunk ID** to the **Chunk IDs** array.
7. The client computes an **Obfuscated File Key** using the **File Key**, the **Base Nonce**, and the **Condensed MAC**:

```
Obfuscated File Key = [
    // XOR first 24 bytes of File Key with Base Nonce
    File Key[0]  $\oplus$  Base Nonce[0],
    :
    File Key[23]  $\oplus$  Base Nonce[23],

    // XOR remaining 8 bytes of File Key with Condensed MAC
    File Key[24]  $\oplus$  Condensed MAC[0]  $\oplus$  Condensed MAC[1],
    :
    File Key[31]  $\oplus$  Condensed MAC[14]  $\oplus$  Condensed MAC[15],

    // Fill up with 24 bytes of Base Nonce
    Base Nonce[0],
    :
    Base Nonce[23],

    // Fill up last 8 bytes with Condensed MAC XORed with itself
    Condensed MAC[0]  $\oplus$  Condensed MAC[1],
    :
    Condensed MAC[14]  $\oplus$  Condensed MAC[15],
]
```

8. The **Obfuscated File Key** is then encrypted using the **Master Key** and the **Base Nonce**:

$$\text{Encrypted File Key} = \text{Enc}(\text{Master Key}, \text{Obfuscated File Key}, \text{Base Nonce})$$

The used nonce is prepended to the encrypted data.

9. The client encrypts the **File Metadata**, which consists of the **Chunk IDs** and other file attributes such as its name, the MIME type, and its size using the **File Key**:

$$\text{Encrypted File Metadata} = \text{Enc}(\text{File Key}, \text{File Metadata}, \text{Nonce})$$

where **Nonce** as a unique nonce. The nonce is prepended to the encrypted data.

10. Finally, the file upload is completed by the client sending a request to the server containing

Encrypted File Key
Condensed MAC
Encrypted File Metadata

Until this point, encrypted chunks may be stored on the server and their size is deducted from the user's storage quota, but a client will only be able to successfully reconstruct files if it knows the chunk ids and their encryption parameters. Only after this last complete upload request, the client can reconstruct files, as this entity contains the stored chunk ids. It may be necessary to perform frequent automatic cleanup checks on the client side to delete unreferenced chunks.

4.4 File Decryption

First, the client has to retrieve the root file entry which consists of the **Encrypted File Key**, the **Condensed MAC**, and the **Encrypted File Metadata**. With their **Master Key**, a user is able to decrypt the **Obfuscated File Key** and de-obfuscate it by passing it into the obfuscation function together with the **Condensed MAC** and the **Base Nonce** (which was prepended to the **Encrypted File Key**).

Then, the **File Metadata** is decrypted using the retrieved **File Key**. This allows the client to identify the chunks of the file, which are then fetched from the backend and decrypted using the **File Key**. Finally, the file is reconstructed by concatenating all decrypted chunks in the correct order.

4.5 Public Vaults

Public vaults make extensive use of asymmetric cryptography using both Ed25519 for authentication and X25519 for exchanging file encryption keys.

4.5.1 Creating A Vault

Upon creation, the client creates a random 32-byte **Vault Key** which is encrypted using the user's **Master Key**. Additionally, the vault metadata (currently only the vault's name) are encrypted using this **Vault Key**.

For each defined slot, its metadata (also only the slot's name for now) are encrypted using the **Vault Key**. The slot's **Maximum Upload Size** remains in plaintext, as this information is needed for the server to limit file uploads. Finally, the client generates an **Ed25519 Private Key** and its corresponding **Ed25519 Public Key**. This private key will be used to authenticate a legitimate third party to access the vault and upload files. The private key is then encrypted using the **Vault Key**.

Finally, the following data is sent to the server to create the vault:

Encrypted Vault Key
Encrypted Vault Metadata
Encrypted Ed25519 Private Key
Ed25519 Public Key

For each slot:

Encrypted Slot Metadata
Maximum Upload Size

The backend will then generate a token for the vault which, together with the vault key, is used to access the vault. The client then generates the URL to the vault:

```
"https://<DOMAIN>/v/" || Vault Token || "#" || Base64(Vault Key)
```

4.5.2 Upload File Into A Slot

In order for an anonymous user to access a vault, they must know both the **Vault Token** and the **Vault Key**.

The server can verify that this anonymous user knows the correct **Vault Key** by sending them the **Encrypted Ed25519 Private Key** and a **Challenge**, which is a random 32-byte value, signed by the server. When accessing the vault, the client then decrypts the **Encrypted Ed25519 Private Key** using the **Vault Key** to obtain the **Ed25519 Private Key**. This private key is then used to compute the **Challenge Signature** of the **Challenge**. The client then sends the computed **Challenge Signature**, together with the **Vault Token** and the original **Challenge** to the backend which can verify that the **Challenge** was indeed generated by the backend and that the anonymous user was able to compute the correct signature, which is verified using the vault's **Ed25519 Public Key**.

After an anonymous user has successfully authenticated themselves, they are given the **X25519 Public Key** of the user owning the vault, the **Encrypted Vault Metadata**, and the slots associated with this vault. For each slot its **Encrypted Slot Metadata**, the **Maximum Upload Size**, a flag whether the slot is still available, and an **Upload Ticket** is returned. The **Upload Ticket** is an encrypted message containing the slot id and the current timestamp, and has to be provided by the client for every request related to a slot (i.e. storing a chunk on the server and finalizing an upload). The **Upload Ticket** is only valid for one hour.

Finally, uploading a file into a slot works exactly the same as specified in Section 4.3, but instead of the user's **Master Key**, a **Shared Encryption Key** is used to encrypt the **File Key** and all other metadata. This **Shared Encryption Key** is generated using X25519 with the vault owner's **X25519 Public Key** and an **Ephemeral X25519 Private Key**, which is generated by the anonymous user's client. Furthermore, the **Ephemeral X25519 Private Key** is used to generate the according **Ephemeral X25519 Public Key**, which is also sent to the server when finalizing the upload, as this key is required by the vault's owner to be able to generate the **Shared Encryption Key** as well.

4.5.3 Retrieve File In A Slot

Since the vault owner's **Master Key** was used to encrypt the **Vault Key**, they can simply decrypt the **Encrypted Vault Key**, which gives them access to **Vault Metadata** and **Slot Metadata**. In order to decrypt files uploaded to slots, the owner computes the **Shared Encryption Key** using their personal **X25519 Private Key** and the **Ephemeral X25519 Public Key** of the uploader, which is then used to decrypt the **File Key**. The **File Key** then gives the owner access to the file and its chunks.

5 Implementation Details

In this section we will cover the most important aspects of the web application developed. The web application developed as part of this work serves more as a proof-of-concept and therefore the focus is less on scalability and production readiness.

5.1 Programming Language

There is only one programming language that can be used for dynamic web applications: JavaScript. However, JavaScript does not offer data type safety, which can have undesirable consequences at runtime. This is why this application is written entirely in TypeScript, which is a superset of JavaScript and allows problems with data types to be detected in an additional “compile” step of the transpiler.

5.2 Framework

As the entire application is to be web-based, it makes sense to use a frontend framework that already covers many areas of web development, such as bundling the application, routing, and state management. A less well-known, but more than suitable framework is SvelteKit, which is based on the Svelte compiler.¹ The authors of this framework describe it as a framework for rapidly developing robust and performant web applications. Since SvelteKit is a full-stack framework, we can implement both the frontend and the backend in the same application.

5.3 API

The communication between the frontend and the backend is backed by gRPC,² which is a library for developing type-safe APIs. This is especially useful when both the frontend and backend are written in TypeScript and are located in the same repository. Unfortunately, gRPC cannot handle binary data and is limited to ASCII. This requires the backend to fallback to regular GET and POST requests for certain endpoints.

5.4 Database

Although most of the application logic happens on the client, the server functions as a remote storage. It is given the user data so that it can be accessed from other devices. This data is persisted on the server in an SQLite³ database and operations on this database are handled by Drizzle ORM⁴, which is a popular choice when working with databases in JavaScript. For the sake of simplicity, also the uploaded file chunks, which are just binary data, are stored in the SQLite database.

5.5 Encryption

While modern browsers already come with built-in support for a variety of cryptographic functions, there is no support for XChaCha20-Poly1305 and Curve25519. Luckily, there are three JavaScript libraries @noble/ciphers, @noble/curves, and @noble/hashes by Paul Miller,⁵ which implement the cryptographic functions required for this application. Such a security-critical library poses a major risk if the underlying encryption functions are not implemented correctly or the code contains backdoors. Audits of such code is crucial. Fortunately, the above mentioned libraries have undergone 6 audits by October 2024.

5.6 Sessions

Sessions make it possible for the server to identify and authenticate users. As is usual for web applications, users receive a cookie containing a signed session identifier after logging in. This cookie authorizes a user to make changes to the data assigned to their account. This cookie is only used to identify the device via which the user has logged in. It is not sufficient to read encrypted files.

¹<https://svelte.dev/> – Write breathtakingly concise components that do minimal work in the browser

²<https://grpc.io/> – A high performance, open source universal RPC framework Get started

³<https://www.sqlite.org/> – Small, fast, self-contained, high-reliability, full-featured, SQL database engine

⁴<https://orm.drizzle.team/> – Next gen TypeScript ORM

⁵<https://paulmillr.com/noble/> – A high-security, easily auditable set of contained cryptographic libraries

To be able to read encrypted files, the client requires the **Master Key** that is decrypted using the **Derived Encryption Key** after login. To avoid the user having to enter their password each time they open the page, the **Master Key** is stored in the browser storage, namely in the IndexedDB,⁶ and is read when the page is opened.

5.7 Content-Defined Chunking

Instead of splitting files into fixed-size chunks, they are split into variable-size chunks based on their content. CDC uses a rolling hash to find boundaries where specific conditions (e.g., a bit pattern) are met. In particular, the FastCDC algorithm is used [13], so that the minimum size of a chunk is 1 KB, the average size is 4 MB and the maximum size is 8 MB. FastCDC uses a so-called Gear hash which achieves 10 times the throughput of Rabin-based methods [14].

5.8 File Reconstruction

Once the chunks of a file have been downloaded and decrypted, the file must be reconstructed and, depending on the user's intent, displayed in the browser or even downloaded. However, due to browser limitations, it is not (yet) possible to stream chunks and save them as a file. This becomes a problem for files bigger than 2 GB or 4 GB (depending on the browser). This is where Service Workers⁷ come in handy, as they allow us to respond to specific requests from the web application with a custom response. In our case, the custom response is a file stream which we can simply write the decrypted chunks to and close once the last decrypted chunk has been processed. With this simple trick we can bypass this browser limitation.

6 Cyber Risk Analysis

In this section, we will take a close look at the application's potential security weak spots. This is an important step for future iterations to figure out the best way to deal with them and keep the system as safe as possible. You can find a more detailed table of cybersecurity concerns listed below, their CVSS rating, and potential counter-measures in the Appendix.

1. Operating system & Web browser

The biggest risk for data confidentiality and integrity is probably the operating system and web browser of the user. An outdated platform introduces a lot of risks, as a compromised system basically allows an attacker to access any resource available on the user's device.

2. Side-channel attacks through the npm⁸ ecosystem

The npm ecosystem has a history of security incidents where malicious or compromised packages cause vulnerabilities. Versions should be audited manually and updated or patched as needed. An even better approach would be to review critical source code from dependencies and copy them directly into the application to reduce reliance on external packages.

3. Access via API endpoints

Because the application has not yet undergone professional security audits, it is difficult to determine whether the existing measures are sufficient to protect against potential threats. Therefore, an attack might be able to gain access to improperly secured API endpoints and compromise data confidentiality and integrity.

4. Browser-based cryptography

Although the cryptography libraries used have undergone audits, performing encryption and decryption in a browser still poses inherent risks. However, it is possible to implement a client for this application in other programming languages, and users are not strictly bound to using the web client.

⁶https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API – Client-side storage for structured data

⁷https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API – Proxy between applications and network

⁸<https://www.npmjs.com/> – Package manager for the JavaScript programming language

5. Weak password policy

Another risk of the current application is its weak password policy. Currently, the user password can be any 8-character string, but the actual strength of the password is not evaluated, e.g., the password “12345678” consists of 8 characters but is easily exploitable.

6. No server-side brute-force protection

The server currently does not limit the number of attempts for any operation, which allows attackers to carry out repeated brute-force attacks without hindrance.

7. No key rotation strategy

The master key is not rotated regularly, and a frequent key rotation policy should be in place to minimize the impact of any potential key compromise.

8. Master key stored in browser storage

Storing the master key in the browser’s storage introduces significant risk. If a malicious npm package manages to execute code within the application, it could access the master key, download and decrypt stored files, and upload them to another server.

9. Potential for malicious file uploads in public vaults

If an adversary gains access to the URL of a public vault, they may upload malicious files that the vault owner could be unaware of, thereby putting the vault owner at risk.

10. Risk of replay attacks in public vaults

The current challenge approach for public vaults is susceptible to replay attacks and must be revised to ensure better protection against unauthorized reuse of **Challenge Signatures**.

7 Conclusion

This report presents the design and implementation of a secure, web-based storage service that combines state-of-the-art cryptographic algorithms with practical client-side functionality. The design, which is based on the already audited and field-tested secure storage service MEGA, ensures high performance and strong security guarantees by employing XChaCha20-Poly1305 for encryption, X25519 for key exchange, and Ed25519 for authentication. Features such as content-defined chunking and public vaults demonstrate how the system can adopt to modern storage and collaboration requirements.

However, the implementation also shows some limitations that need to be addressed for production readiness. These limitations include scalability constraints, lack of automatic cleanup of orphaned data, and the absence of robust server-side protections. Other important aspects which are uncovered so far are security audits, key rotation strategies, and enhancements the design of public vaults to mitigate potential risks. The cybersecurity risk analysis conducted as part of this work highlights the benefit of potential counter-measures.

Despite these challenges, I hope that this work provides a good foundation for developing secure and user-friendly encrypted storage services and ideally contributes to the broader discourse on cybersecurity.

7.1 Limitations

Due to the short development time, the web application has some strict limitations.

1. Single root folder

The application currently supports only one root folder without any directory or subfolder structure. This design significantly limits scalability and makes it difficult to organize and manage large numbers of files efficiently.

2. Single SQLite database for all data

Relying on a single SQLite database to store records and binary chunks can limit performance and scalability, especially under high concurrency or large data loads. As the system grows, frequent read/

write operations may slow down, and the risk of database corruption or inefficient data retrieval increases.

3. Coupling between frontend and backend

As the frontend and the backend are both based on the same framework, and even worse, use gRPC for communication, both implementations are strongly coupled. This limits flexibility and scalability of the application, as it makes independent development and integration with other systems more difficult.

7.2 Future Work

Security audits remain a key priority for this application. By engaging professional third-party auditors or performing in-depth internal reviews by following cybersecurity principles, we can evaluate the security of the application more thoroughly and uncover otherwise hidden vulnerabilities. In addition to these audits, other server-side hardening measures should be implemented, such as rate limiting requests.

Another focus is implementing a reliable key rotation mechanism. Rotating encryption keys regularly can significantly reduce the impact of potential breaches, when compromised keys become useless after their rotation period. This also goes hand in hand with better session management in order to give users the ability to monitor and terminate active sessions manually.

It would also be beneficial to add configurable expiration dates to public vault links, to automatically revoke access after a set period. Also, it would be nice to allow the user to rotate the public vault token. At the same time, support for sharing direct links to individual files would improve the collaborative aspect of this application.

Lastly, automatic cleanup of unreferenced chunks would help maintain the efficiency of the storage system. By periodically scanning and removing data not longer in use, the application can conserve disk space, reduce clutter, and minimize potential security risks posed by forgotten or outdated files.

References

- [1] K. Manheim and L. Kaplan, “Artificial intelligence: Risks to Privacy and Democracy,” *Yale JL & Tech.*, vol. 21, p. 106, 2019.
- [2] Statista, “Cyber Crime & Security.” Accessed: Jan. 07, 2025. [Online]. Available: <https://www.statista.com/markets/424/topic/1065/cyber-crime-security/>
- [3] R. A. Kemmerer, “Cybersecurity,” in *25th International Conference on Software Engineering, 2003. Proceedings.*, 2003, pp. 705–715. doi: 10.1109/ICSE.2003.1201257.
- [4] N. Ferguson, B. Schneier, and T. Kohno, *Cryptography Engineering: Design Principles and Practical Applications*. John Wiley & Sons, 2011.
- [5] S. Arciszewski, “XChaCha: eXtended-nonce ChaCha and AEAD_XChaCha20_Poly1305,” Internet Engineering Task Force, Jan. 2020. [Online]. Available: <https://datatracker.ietf.org/doc/draft-irtf-cfrg-xchacha/03/>
- [6] Y. Nir and A. Langley, “ChaCha20 and Poly1305 for IETF Protocols.” [Online]. Available: <https://www.rfc-editor.org/info/rfc8439>
- [7] MEGA, “MEGA Security Whitepaper - Third Edition.” Accessed: Oct. 30, 2024. [Online]. Available: <https://mega.nz/SecurityWhitepaper.pdf>
- [8] OWASP, “Password Storage Cheat Sheet.” Accessed: Jan. 03, 2025. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html#pbkdf2
- [9] A. J. Menezes, P. C. Van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*. in Discrete Mathematics and Its Applications. CRC press, 2018.

- [10] N. R. Council *et al.*, *Cryptography's Role in Securing the Information Society*. National Academies Press, 1996.
- [11] B. Schneier, *Applied Cryptography: Protocols, Algorithms and Source Code in C*. Wiley, 2015.
- [12] S. Contini, “Method to Protect Passwords in Databases for Web Applications.” [Online]. Available: <https://eprint.iacr.org/2015/387>
- [13] W. Xia *et al.*, “FastCDC: A Fast and Efficient Content-Defined Chunking Approach for Data Deduplication,” in *2016 USENIX Annual Technical Conference (USENIX ATC 16)*, Denver, CO: USENIX Association, Jun. 2016, pp. 101–114. [Online]. Available: <https://www.usenix.org/conference/atc16/technical-sessions/presentation/xia>
- [14] W. Xia *et al.*, “The Design of Fast Content-Defined Chunking for Data Deduplication Based Storage Systems,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 9, pp. 2017–2031, 2020, doi: 10.1109/TPDS.2020.2984632.

Appendix

1. Analyze and identify vulnerabilities and threats							2. Estimate and evaluate risk			
ID	System Overview/Context View	Vulnerability	STRIDE Category	Threat	Asset	Impact	CVSS Base Score	CVSS Vector String	CVSS Rating	Risk
1	Operating System	Outdated OS vulnerabilities	Spoofing, Tampering, Elevation of Privilege	Unauthorized access via unpatched OS exploits	System & Data	Potential full system compromise	9.8	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H	Critical	High
2	npm Ecosystem Dependencies	Malicious package injection	Tampering, Information Disclosure	Third-party dependency compromise	Application Code	Unauthorized system access	9.8	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H	Critical	High
3	API Endpoints	Excessive API permissions	Elevation of Privilege	Users might perform unauthorized actions via misconfigured API	Application Logic	Unauthorized modification of system state	8.3	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L	High	High
4	Browser-Based Cryptography	Potential flaws in cryptographic library	Tampering	Weaknesses in cryptographic operations leading to exposure	Encrypted Data	Data compromise due to weak cryptographic implementation	8.2	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N	High	High
5	User Authentication	Weak password policy	Elevation of Privilege	Users may set weak passwords, increasing brute-force success	User Accounts	Unauthorized access to user data	8.2	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N	High	High
6	Authentication Mechanism	No brute-force protection	Elevation of Privilege	Credential stuffing attacks	User Accounts	Unauthorized access to user accounts	7.5	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N	High	High
7	Cryptographic Key Management	No key rotation strategy	Tampering	Compromised encryption keys remaining valid indefinitely	Encryption Keys	Long-term exposure of encrypted data	8.2	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N	High	High
8	Web Storage Service	Master key stored in browser storage	Information Disclosure, Tampering	Key exposure through malicious scripts	Encryption Keys	Unauthorized decryption of files	9.8	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N	Critical	High
9	Public Vaults	Malicious file uploads	Information Disclosure, Spoofing	Attackers may upload malicious files leading to compromise	User Files	Execution of malicious files or phishing attempts	8.3	AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:L	High	High
10	Public Vault Authentication	Risk of replay attacks	Spoofing	Attackers replay authentication tokens to gain unauthorized access	Vault Access	Unauthorized access to uploaded files	8.2	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N	High	High

3. Control security risk			4. Assess risk after mitigation			
ID	Security Risk Mitigation Specification	Verification of Security Risk Controls	CVSS Base Score	CVSS Vector String	CVSS Rating	Risk
1	Regular OS updates, vulnerability scanning	Check patch logs and vulnerability scan reports	6.3	AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:L/A:L	Medium	Reduced
2	Perform manual audits of critical dependencies, pin versions	Verify SBOM and pinned versions	5.4	AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:L/A:N	Medium	Reduced
3	Implement least privilege access control, review API permissions	Check role-based access and run permission tests	4.2	AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:L/A:N	Medium	Reduced
4	Rely on well-audited libraries, validate cryptographic integrity	Check cryptographic audits and library integrity	4.3	AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N	Medium	Reduced
5	Enforce strong password policies, require MFA	Check MFA logs and validate password complexity settings	3.7	AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N	Low	Reduced
6	Implement rate limiting, CAPTCHA, account lockout	Inspect rate-limit triggers and lockout logs	3.7	AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N	Low	Reduced
7	Implement periodic key rotation with re-encryption	Check key rotation logs and verify re-encryption process	4.3	AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N	Medium	Reduced
8	Use secure enclave or hardware security module for key storage	Check hardware key management logs and local storage use	5.4	AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:L/A:N	Medium	Reduced
9	File type validation, antivirus scanning, user confirmations before execution	Check file scanning logs and file type validations	5.4	AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N	Medium	Reduced
10	Implement nonce-based authentication challenges	Verify replay detection and time-limited token settings	3.7	AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N	Low	Reduced

Figure 1: Cyber-risk analysis for the Web-based, End-to-end Encrypted Secure Storage Service. For 10 possible vulnerabilities and threats, the impact on the system is assessed and measured using the Common Vulnerability Scoring System (CVSS) v3.1. Additionally, possible mitigations are explored and their impact on the security is measured using CVSS.